

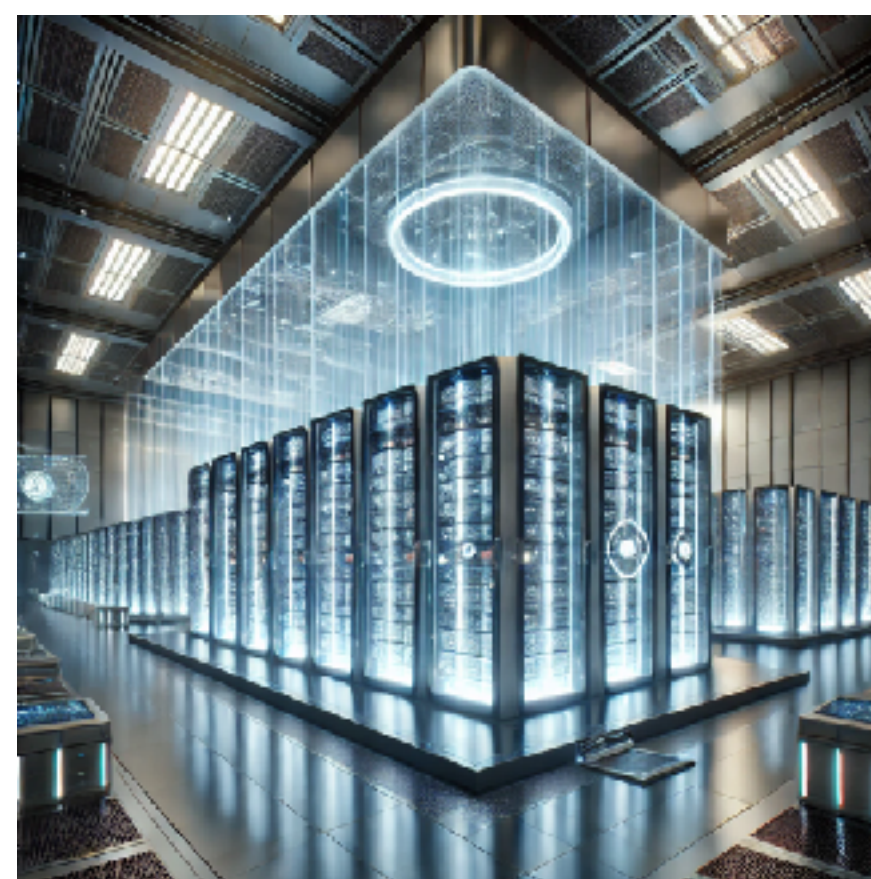
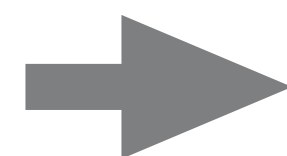
大規模言語モデルの訓練方法（詳細；講義ではスキップ）

ステップ1：事前学習（Pretraining）

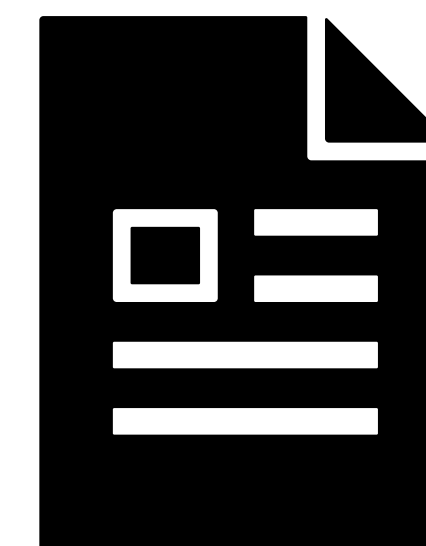
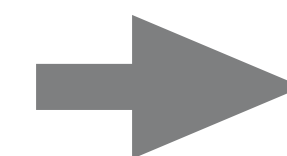
大量のデータから大量の計算量を使って学習



~15兆トークン
(12兆単語程度)
のテキストデータ



延べ3,084万時間の計算
(1台600万円近くする
GPUを16,000台使用)



params.zip

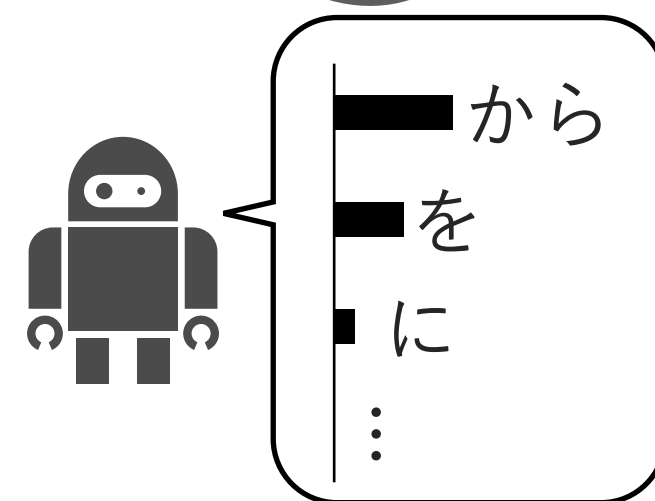
229GB

まるでイン
ターネットを
圧縮したよう
なもの

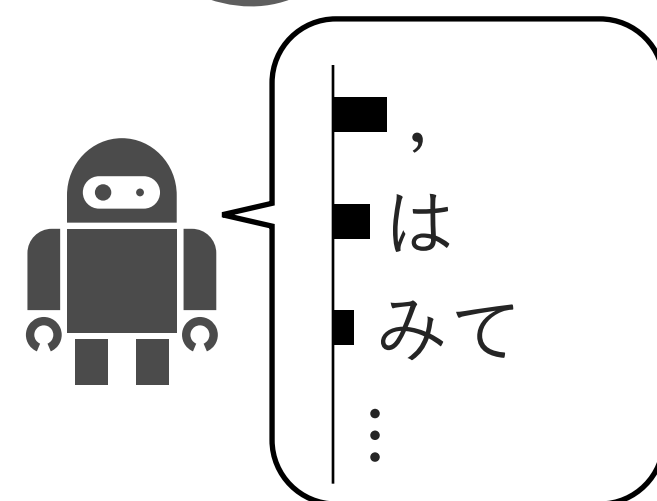
事前学習 > 訓練方法

次トークン予測（next-token generation）による自己教師付き学習

- 途中までの文章 [?]，次のトークンを予測させる



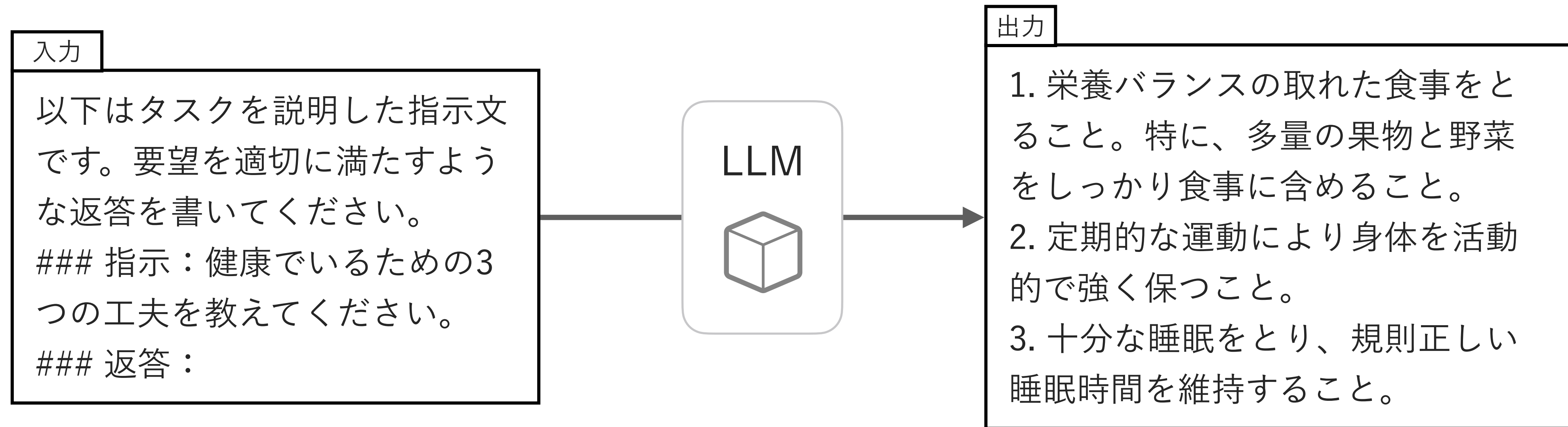
- 途中までの文章から [?] 次のトークンを予測させる



ステップ2：指示チューニング（Instruction tuning）

言語モデルをエージェントらしく挙動させるために教師付き追加訓練

- 指示と正解出力のペアで、教師付き精密調整（ファインチューニング）をする



指示チューニング>データセット

指示チューニング用のデータセット例（Taori et al., 2023）

instruction string · lengths	input string · lengths	output string · lengths	text string · lengths
 9↔58 51.7%	 0↔247 98.8%	 0↔419 74.5%	 154↔589 67.2%
Give three tips for staying healthy.		1.Eat a balanced diet and make sure to include plenty of fruits and vegetables. 2. Exercise regularly to keep your body active and strong. 3. Get enough sleep and maintain a consistent sleep schedule.	Below is an instruction that describes a task. Write a response that appropriately completes the request. ### Instruction: Give three tips for staying healthy. ### Response: 1.Eat a balanced diet and make sure to include plenty of fruits and vegetables. 2. Exercise regularly to keep your body active and strong. 3. Get enough sleep and maintain a consistent sleep schedule.
What are the three primary colors?		The three primary colors are red, blue, and yellow.	Below is an instruction that describes a task. Write a...
< Previous 1 2 3 ... 521 Next >			

【参考】 Taori, R., Gulrajani, I., Zhang, T., Dubois, Y., Li, X., Guestrin, C., Liang, P., & Hashimoto, T. B. (2023). Stanford Alpaca: An Instruction-following LLaMA model [Dataset]. stanford_alpaca.

指示チューニング>データセット

どの程度の指示チューニングデータで学習が行われているか？

Model	# of Prompts	Source
Llama 2	28K	Touvron et al. (2023)
InstructGPT	15K	Ouyang et al. (2022)
Alpaca	52K	Taori et al. (2023)
Vicuna	70K	Chiang et al. (2023)
Dolly	15K	Conover et al. (2023)
OpenAssistant	10K+	Köpf et al. (2023)
Claude	137K + 369K	Bai et al. (2022)
WizardLM	624K	Xu et al. (2023)
LIMA	1K	Zhou et al. (2023)

【出典】 Ustalov, D., & Lambert, N. (2023, July 20). Reinforcement Learning from Human Feedback: A Tutorial [Tutorial]. The Fortieth International Conference on Machine Learning, Honolulu. <https://icml.cc/virtual/2023/tutorial/21554>

指示チューニング>データセット

指示チューニング用データはどうやって集めるのか？

- 既存の自然言語処理のデータセットを指示チューニングに転用できる



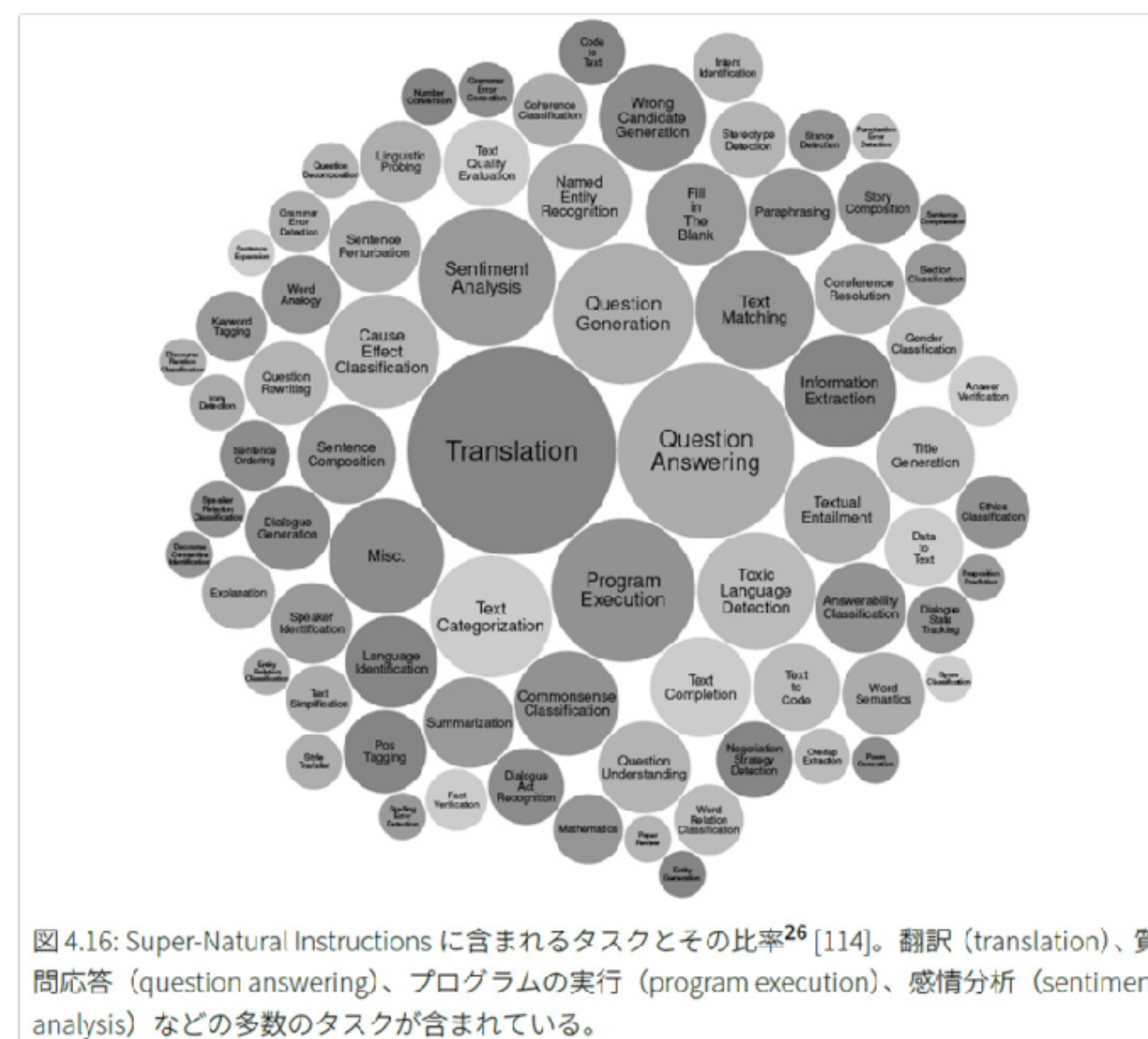
図 4.15: 質問応答・自然言語推論データセットからの指示チューニングデータセットの作成²⁵ [115]。テンプレート枠内の上側がプロンプト、下側が出力テキストを表す。{ フィールド名 } のように示した部分が該当するフィールドの値で置き換えられる。

【図】 山田育矢, 鈴木正敏, 山田康輔, & 李凌寒. (2023). 大規模言語モデル入門. 技術評論社. の図4.15より抜粋.

指示チューニング>データセット

テンプレート生成で、大規模な指示チューニング用データが構築されている

- Natural Instructions, Super-Natural Instructions, P3 (Public Pool of Prompts)



指示チューニング>データセット

指示チューニング用データセットを複数連結して更に大規模に

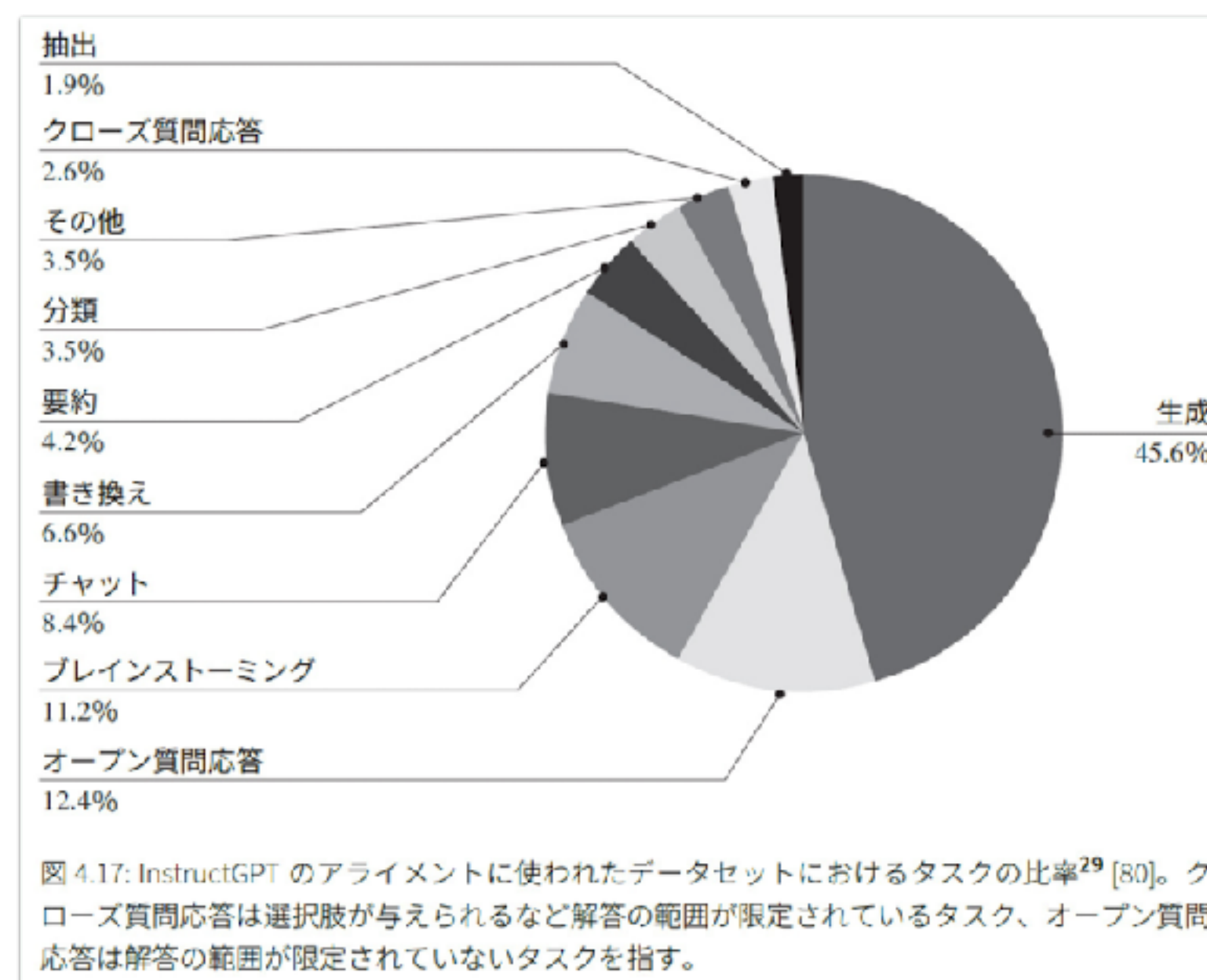
- Flan 2022 Collection

- FLANで用いられたデータセット, Super-Natural Instructions, P3を連結
- CoT推論や対話などの事例も追加されている

指示チューニング>データセット

人手での指示チューニング用データセット作成も行われている

- InstructGPTでは、15,000件程度のデータセットが作成された
 - 入力：自前のプロンプト + 初期のInstructGPTのAPIに送られてきたプロンプト
 - 理想的な出力：人手で付与（40人がオンラインで雇われて実施）



指示チューニング>限界

指示チューニングデータを大規模かつ高品質に作るのは難しい

- 既存データセットの再利用の場合，テンプレートを使うので出力の多様性が低い
- 人手での作成の場合，出力自体を人が作らなければならないので高コスト
- ただし，厳選された1,000件のデータセットによる指示チューニングで，RLHFや選好モデリングもせずに，InstructGPTに匹敵する性能を達成したという報告もある（Zhou et al., 2023）ので，そもそも指示チューニングを大規模データで行う必要がない可能性はある

指示チューニング>限界

出力を書くのが難しいケースもある

- 創造的な生成タスクの場合

- 「小説を書いてください」

- 回答を棄権させたい場合

- モデルが質問の答えを事前学習した知識として知っている→回答をする
- モデルが質問の答えを事前学習した知識として知らない→回答を棄権する
- しかし、モデルが知識を保持しているかどうかで分岐するのは現実的でない

ステップ3：選好チューニング

まずモデルに出力させてみて、それに対する人間の選好で学習

- さらに人に好まれる回答をするようにアシスタントとしての訓練
 - 有害な回答・バイアスなどのアラインメント的な側面もカバー
- 色々な手法が用いられる
 - 人のフィードバックからの強化学習（reinforcement learning from human feedback, RLHF）
 - RLHFから派生した直接選好最適化（direct preference optimization, DPO）

選好チューニング > 指示チューニングとの比較

指示チューニングで扱いづらい部分を補うことができる

	指示チューニング	選好チューニング
創造的なタスク (例: 小説を書く)	あらかじめ小説を書く 必要がある	モデルが出力した小説 の良し悪しを評価する だけ
細かい内容や表現	事前にバリエーション を用意しておく必要が ある	モデルの表現を比較評 価するだけ
特定の問題ある 出力の抑制	特に良い打ち手がな い？	その特定の出力を抑制 するようにフィード バックしやすい

RLHF > 全体の流れ

選好データを収集 → 報酬関数を推定 → 報酬最大化によりモデルを訓練

選好チューニングしたい生成モデル： $\pi_{\text{ref}}(y|x)$

- 1) 人の選好データを収集：同じ入力 x に対する π_{ref} の複数の出力をランキングしてもらう
- 2) 選好データから報酬関数 $r(x, y)$ を推定（入力 x に対する出力 y の良し悪しのスコア）
- 3) 報酬関数に少し手を加える（乖離ペナルティを組み込む）

$$\tilde{r}_{\theta}(x, y) := r(x, y) - \beta \log \left(\frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)} \right)$$

- 4) 報酬を最大化するようにファインチューニング： $\pi_{\text{ref}}(y|x) \rightarrow \pi_{\theta}(y|x)$

$$\theta_{t+1} \leftarrow \theta_t + \alpha \frac{1}{n} \sum_{i=1}^n \tilde{r}_{\theta_t}(x_i, y_i) \nabla_{\theta_t} \log \pi_{\theta_t}(y_i | x_i)$$

選好チューニング>データ収集

アノテーターは、まずLikertスケール（1-7）で評価し、メタデータを付加

Submit

Skip

«Page 3 / 11»

Total time: 05:39

Instruction

Summarize the following news article:

====
{article}
====

Include output

Output A

summary1

Rating (1 = worst, 7 = best)

1

2

3

4

5

6

7

Fails to follow the correct instruction / task ?

☐ Yes

☐ No

Inappropriate for customer assistant ?

☐ Yes

☐ No

Contains sexual content

☐ Yes

☐ No

Contains violent content

☐ Yes

☐ No

Encourages or fails to discourage violence/abuse/terrorism/self-harm

☐ Yes

☐ No

Denigrates a protected class

☐ Yes

☐ No

Gives harmful advice ?

☐ Yes

☐ No

Expresses moral judgment

☐ Yes

☐ No

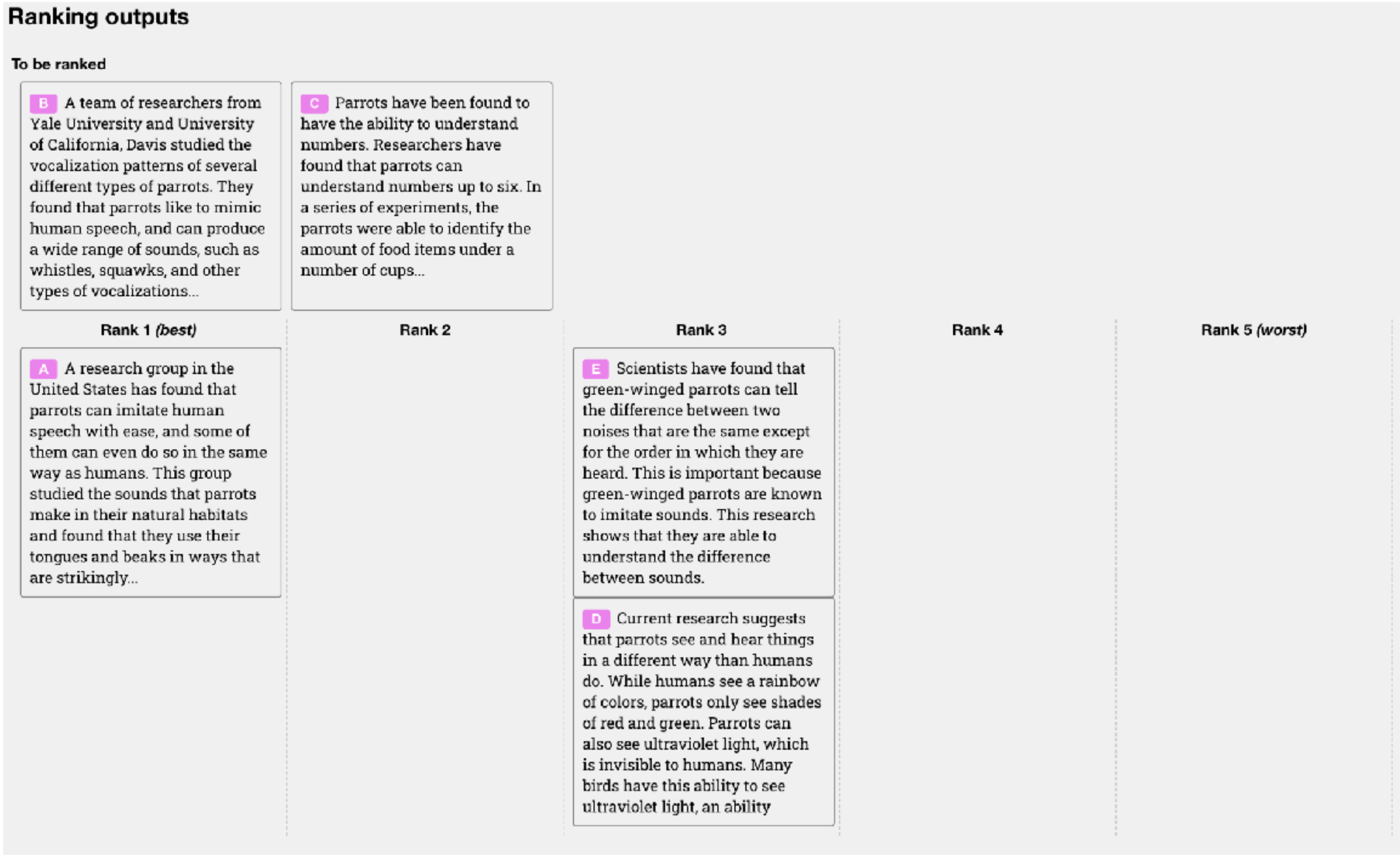
Notes

(Optional) notes

【図】 Figure 12 of Ouyang et al. (2022). Training language models to follow instructions with human feedback. Advances in Neural Information Processing Systems, 35, 27730–27744. 【補足】 InstructGPTの論文では、 UpworkとScaleAIというサービスで40人を採用。特に事前にスクリーニングテストを実施。

選好チューニング > データ収集

続いて K 個の文章をランク1～5に振り分けることにより比較 ($4 \leq K \leq 9$)



【図】 Figure 12 of Ouyang et al. (2022). Training language models to follow instructions with human feedback. Advances in Neural Information Processing Systems, 35, 27730–27744.

RLHF > 報酬モデルの訓練

絶対評価を与える報酬モデル $r(x, y)$ を，相対評価である選好データから推定

- 報酬モデル $r_\psi(x, y)$ から選好モデル（Bradley-Terry model）を構成

$$p_\psi(y_A \succ y_B | x) = \sigma(r_\psi(x, y_A) - r_\psi(x, y_B))$$

- 選好モデルとしての最尤推定で学習（ $\mathcal{D} := \{(x_i, y_i^{(\text{winner})}, y_i^{(\text{loser})})\}_{i=1}^n$ 選好データ）

$$L(\psi) := \hat{\mathbb{E}}_{\mathcal{D}}[-\log p_\psi(Y_{\text{winner}} \succ Y_{\text{loser}} | X)] \rightarrow \text{最小化}$$

【補足】ここで σ は標準ロジスティック関数 $\sigma(s) := 1/(1 + \exp(-s))$ 。実際は1つの x に対して K 個の y を一度に生成して選好データを収集している（ K は4から9）。報酬モデルの学習時も $\{r_\psi(x, y_k)\}_{k=1}^K$ の全通りの組み合わせを一度に扱って平均して， $\hat{\mathbb{E}}_{\mathcal{D}}[(K C_2)^{-1} \sum_{w,l: y_w \succ y_l} -\log \sigma(r_\psi(x, y_w) - r_\psi(x, y_l))]$ を最小化するように学習している。

【参考】Ouyang et al.(2022). Training language models to follow instructions with human feedback. NeurIPS, 35, 27730–27744.

RLHF > モデルのファインチューニング

報酬モデルを得たあとは、報酬モデルの値を最大化するように精密調整

- 次の値の最大化を目指してLLM $\pi_{\theta}(y|x)$ をファインチューニング

$$\mathcal{L}(\theta) := \underbrace{\mathbb{E}_{X \sim \mathcal{D}, Y \sim \pi_{\theta}(\cdot|X)}[r(X, Y)]}_{\text{報酬モデル値を最大化}} - \underbrace{\beta \text{KL}(\pi_{\theta} \parallel \pi_{\text{ref}})}_{\text{乖離を抑制}}$$

- $r(x, y)$: 訓練済みの報酬モデル, $\pi_{\text{ref}}(y|x)$: 精密調整前のLLM

- 最適化手法

- 期待値記号自体にパラメーターが入っているので、ひと工夫が必要

RLHF > 方策の訓練

工夫は1つ. 対数の微分のトリックを使う : $(\log x)' = \frac{1}{x}$

- logの微分の性質から $\nabla_{\theta} \pi_{\theta} = \pi_{\theta} \cdot \nabla_{\theta} (\log \pi_{\theta})$ と書き直せるので,

$$\begin{aligned}\nabla_{\theta} \mathbb{E}_{\pi_{\theta}}[f(Z)] &= \nabla_{\theta} \int \pi_{\theta}(z) f(z) dx dy \\ &= \int \pi_{\theta}(z) (\nabla_{\theta} (\log \pi_{\theta}(z))) f(z) dx dy \\ &= \mathbb{E}_{\pi_{\theta}} [(\nabla_{\theta} \log \pi_{\theta}(Z)) f(Z)]\end{aligned}$$

- 右辺のサンプル近似により, 期待値記号にパラメーターが入っている項の勾配を推定

$$\theta \leftarrow \theta + \alpha_t \cdot \left(\frac{1}{n} \sum_{i=1}^n \nabla_{\theta} \log \pi_{\theta}(z_i) \cdot f(z_i) \right) \quad (\{z_i\}_{i=1}^n \stackrel{\text{i.i.d.}}{\sim} \pi_{\theta})$$

【補足】 このトリックを使うことによる報酬最大化は, REINFORCE (モンテカルロ方策勾配) という強化学習法に相当する. 強化学習で最適化するというと大袈裟だが, なんのことはない, 1エピソードあたり1ステップしかないので状態遷移がなく, 最もシンプルな場合になっている.

RLHF > 方策の訓練

先ほどのトリックをもとにして，以下のアルゴリズムで報酬最大化を目指す

■ REINFORCE（モンテカルロ方策勾配法）によるRLHF

- 予め報酬モデルに手を加えておく（乖離ペナルティ項を報酬値に組み込む）

$$\tilde{r}_{\theta}(x, y) := r(x, y) - \beta \log \left(\frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)} \right)$$

- 以下を繰り返す（モンテカルロ方策勾配法）：

- ▶ 最新のパラメーターでサンプルを生成： $\{(x_i, y_i)\}_{i=1}^n$

- ▶ 勾配方向にパラメーターを更新 $\theta \leftarrow \theta + \alpha_t \cdot \left(\frac{1}{n} \sum_{i=1}^n \nabla_{\theta} \log \pi_{\theta}(y_i|x_i) \cdot \tilde{r}_{\theta}(x_i, y_i) \right)$

【補足】ここで，前項でいう f が θ に依存しているので，本当は積の微分をとらないと $\mathcal{L}$ の勾配にならないが，どうやら実践的には f の中の θ は固定してアルゴリズムが実行されている。つまり，KLダイバージェンスの項は，対数勾配ベクトルの重みづけだけに用いられているようだ（実装例：https://github.com/vwxyzjn/lm-human-preference-details/blob/ccc19538e817e98a60d3253242ac15e2a562cb49/lm_human_preference_details/train_policy_accelerate.py#L630）。

RLHF > 方策の訓練（実際）

勾配だけでは更新すべき方向しか分からない → 適切なステップサイズは？

- 信頼領域方策最適化（trust-region policy optimization, TRPO）が答えの一つ.
- 近接方策最適化（proximal policy optimization, PPO）はTRPOの改良版.
- いずれも、精度が悪いかもしれない方策勾配推定量を使いながら適切なステップ幅で更新するためには？という問いへの答え
- 実際には先ほどのREINFORCEではなく、PPOがよく用いられている

RLHF > 方策の訓練（実際）

実際には更に、事前学習の内容を忘れないようにする損失項も組み込まれる

- アラインメントを行うことによる性能劣化（alignment tax）
 - RLHFを実行することで、事前学習タスクの内容を忘れてしまう
 - そうならないように、事前学習の次トークン予測の報酬も同時に最適化

$$\mathcal{L}(\theta) + \gamma \mathbb{E}_{x \sim \text{Dataset}} [\log \pi_{\theta}(x)]$$

RLHF > 課題

RLHFでは報酬関数の推定がある分だけ，訓練の難易度が高い

- 報酬関数はうまく推定できている？
 - 選好データの品質が高くても，報酬関数がうまく推定できていなければ無意味
- 報酬関数のモデルをどう設計する？
 - ハイパーパラメーターが増えてしまう

直接選好最適化 (Direct Preference Optimization, DPO)

報酬モデリングをバイパスする

- 直接選好最適化 (Direct Preference Optimization, DPO)
 - 次の目的関数で方策モデル自体を学習

$$L(\theta) := - \mathbb{E}_{\text{Dataset}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(Y_{\text{winner}} | X)}{\pi_{\text{ref}}(Y_{\text{winner}} | X)} - \beta \log \frac{\pi_{\theta}(Y_{\text{loser}} | X)}{\pi_{\text{ref}}(Y_{\text{loser}} | X)} \right) \right]$$



DPO > 導出

報酬モデリングをバイパスする

- RLHFは、報酬モデル $r(x, y)$ と、参照方策 $\pi_{\text{ref}}(y|x)$ から方策 $\pi^*(y|x)$ を得る操作

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{X \sim \mathcal{D}, Y \sim \pi(\cdot|X)}[r(x, y)] - \beta \text{KL}(\pi \| \pi_{\text{ref}})$$



- 実は、最適な $\pi^*(y|x)$ は解析的に求められる

$$\pi_{Y|X}^*(\cdot | \cdot) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \cdot \exp\left(\frac{r_{X,Y}(x, y)}{\beta}\right)$$

DPO > 導出

報酬モデリングをバイパスする

- 逆算すると, 「 π^* が最適解として出力されるような」 報酬関数 r が一意に定まる

$$\pi^*(y|x) \propto \pi_{\text{ref}}(y|x)e^{(1/\beta)r(x,y)} \quad \Leftrightarrow \quad r(x,y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} (+\text{const.})$$

- これを r の推定の目的関数に代入すれば, 元の「 r の最尤推定」→「 π^* の推定」という2段階をまとめて, 直接的に π^* の教師シグナルとして最尤法で訓練できる



DPO > RLHFとの関係

RLHFの難点を回避して，手法として扱いやすくなっている

- RLHFと比べて，報酬モデルの推定を迂回しているので，訓練しやすい
 - 報酬モデルの分ハイパーパラメーターが減っている
- なお，本当に同じ目的関数を最適化していると言ってよいかどうかは疑問が残る
 - RLHFの手法では，KLダイバージェンス項の θ の勾配が無視されている
 - DPOでは，KLダイバージェンス項にあたる部分の勾配の寄与も（解析的な式変形を通して）あるはず